

TP-DSP- Filtrage RII en temps réel

Lukas Maël

Archenault Maël

Introduction

Ce TP a pour objectif la réalisation d'un système de traitement du signal en temps-réel. L'ensemble du projet se déroulera sur une carte EZ-KIT Lite 21262 de chez Analog Devices. Cette carte inclut un processeur ADSP21262 d'architecture SHARC, et de nombreux périphériques utiles au traitement du signal.

1. Interruption IRQ_P1

Dans cette première partie, nous allons mettre en place une interruption. Celle-ci sera causée par un signal externe.

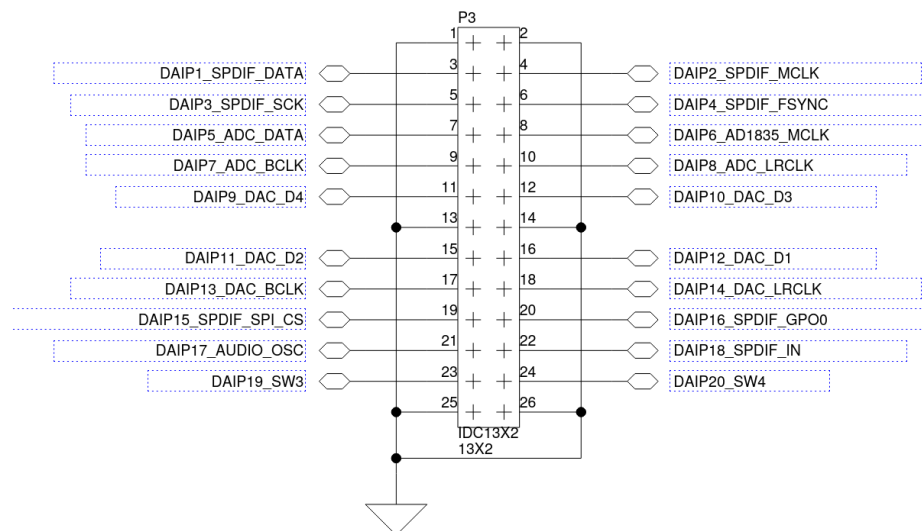
Branchement

Pour l'instant, sur la carte sont connectés :

- le câble d'alimentation
- le câble USB pour la connexion avec l'IDE

Nous allons rajouter une autre connexion qui enverra le signal d'interruption. L'interruption utilisera la broche DAIP1.

On peut trouver sur la datasheet de la carte que cette broche est la broche numéro 3.

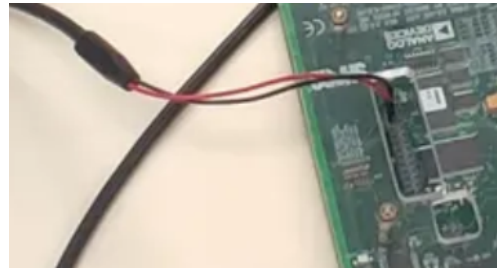


Remarque : Attention, il ne faut pas oublier de relier la masse à la carte. Cela peut se faire via les broches 1,2,13,14,25 et 26. On utilisera la broche 1.

On utilise un GBF pour générer le signal d'interruption. Utiliser le signal "sync" du GBF suffit. On garde la sortie principale du GBF libre pour la suite.



Branchement sur le GBF



Branchement sur la carte

Code

On commence par inclure les lignes suivantes (données sur le sujet du TP).

```
*pDAI_IRPTL_PRI = SRU_EXTMISC1_INT; // Démasquer individuellement l'interruption
*pDAI_IRPTL_RE = SRU_EXTMISC1_INT; // Choisir le front montant pour l'interruption
// Unité de routage : Router (DAI_P1 ----> MISC1)
SRU(LOW,DAI_PB01_I);
SRU(DAI_PB01_O,MISC1_I);
SRU(LOW,PBEN01_I);
```

Ces lignes servent à :

- activer l'interruption sur front montant
- relier le pin externe DAI_PB01 à MISC1, qui est lui-même relié à l'interruption

On utilise la fonction "interrupt" pour définir le callback de l'interrupt.

```
interrupt(SIG_DAIH,IRQP1_isr_leds);
```

Dans notre cas, on veut simplement faire clignoter les leds, la fonction de callback est donc la suivante :

```
void IRQP1_isr_leds (int sig_num)
{
    static int IT_register;
    IT_register = *pDAI_IRPTL_H;

    //tester si c'est bien DAI_P1 qui est à l'origine de l'interruption DAI
    if ((IT_register & SRU_EXTMISC1_INT) != 0)
    {
        AfficheLEDs(255*state);
        state = (state + 1)%2;
    }

    return;
}
```

- Vérification de l'origine de l'interruption
- Changement d'état des 8 leds à chaque appel de l'interruption

Remarque : La fonction "AfficheLEDs" a été récupérée dans les ressources du TP4.

```
void AfficheLEDs(int led_value){
    if (led_value > 255){
        led_value = 255;
    }
    *pPPCTL=0;

    *pIIPP=(int) &led_value;
    *pIMPP=1;
    *pICPP=1;
    *pEMPP=1;
    *pECPP=1;
    *pEIPP=0x1400000;

    *pPPCTL=PPTRAN | PPBHC | PPDUR20 | PPDEN | PPEN;
}
```

On peut donc maintenant compiler le code et l'envoyer sur la carte.

Les leds se mettent alors à clignoter.

2. Validation du filtre en simulation

Cette partie a été faite lors des tutoriels.

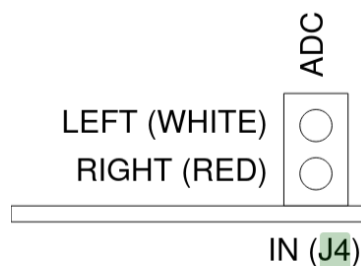
3. Application "Talk-Through"

Notre objectif ici, est d'obtenir un processeur capable de restituer un signal qui lui est passé en entrée. C'est la première étape avant de vouloir appliquer un traitement au signal.

Branchements

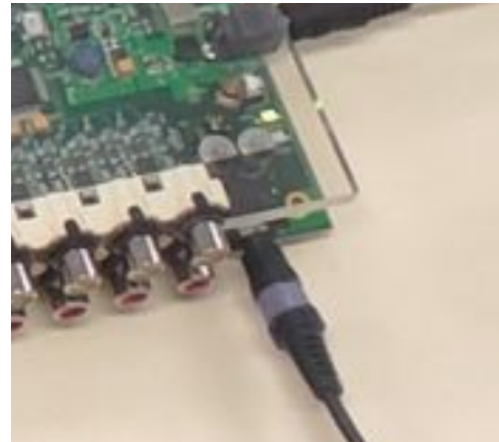
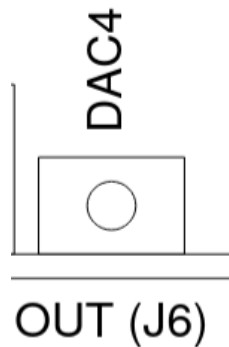
Nous aurons maintenant besoin d'envoyer un signal en entrée de la carte. On utilise donc la sortie du GBF pour générer un signal sinusoïdal (on envoie aussi ce signal dans l'oscilloscope).

On envoie ce signal sur l'entrée de l'ADC de la carte (via le connecteur RCA J4)



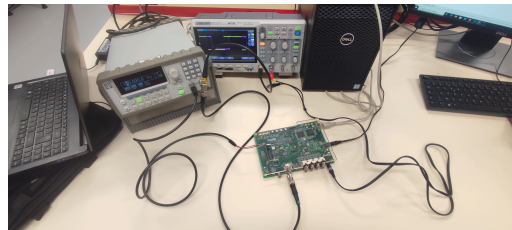
Remarque : Comme il y a deux entrées, on choisit d'utiliser la borne rouge. En sortie, on prendra donc aussi le signal sortant de la borne rouge.

On branche la sortie de la carte (port jack) sur l'oscilloscope



De cette manière, on est capable de voir à l'oscilloscope :

- le signal d'entrée
- le signal de sortie (qui a été traité par le processeur)



Code

Tout le code pour cette partie nous est fourni sur Moodle dans les fichiers main.c, initSPORT.c, initDAI.c et init1835viaSPI.c.

Ceux-ci mettent en place :

- les interruptions effectuées par l'ADC pour signifier l'arrivée d'un échantillon
- la lecture des échantillons en entrée
- la retransmission de ces échantillons sur la sortie

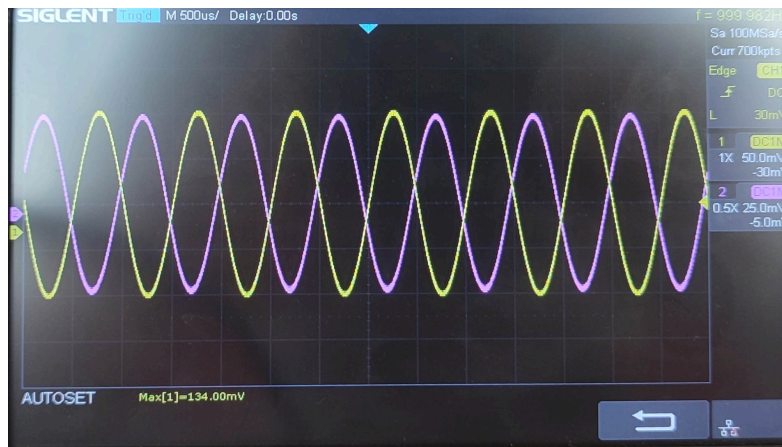
```
void ReceptionISR(int sig_int)
{
    tx_buf[0]=rx_buf[0];
    tx_buf[1]=rx_buf[1];
}
```

Remarque :

- rx → réception
- tx → transmission
- le signal est en stéréo, il y a deux signaux en parallèle. Chaque échantillon est donc composé de deux valeurs

Résultat :

Après compilation du code et envoi sur la carte, on peut observer les signaux suivants à l'oscilloscope :



- En jaune : signal envoyé dans le processeur
- En rose : signal reçu du processeur

Remarque : Le signal de sortie est en retard, sur le signal d'entrée. Ce retard est dû au temps de conversion analogique→numérique puis numérique→analogique.

Traitement basique du signal

Dans une première approche du traitement du signal par le processeur, on veut réussir à diviser par 2 l'amplitude du signal d'entrée.

La solution suivante ne fonctionnera pas :

```
tx_buf[0]=(int)((float)rx_buf[0]/2);|
tx_buf[1]=(int)((float)rx_buf[1]/2);
```

En fait, les valeurs renvoyées par l'ADC sont des entiers signés sur 24 bits. En C, les entiers sont sur 32 bits. Alors quand on applique la division, le code lit la valeur, mais signée sur le mauvais nombre de bits, puis applique la division.

Le résultat du calcul n'a donc rien à voir avec la moitié du signal d'entrée.

La solution est de d'abord transformer l'entier 24 bits en entier 32 bits. Pour faire ça, on utilise une "extension de signe".

```
rx_buf[0] = (rx_buf[0]<<8)>>8;
rx_buf[1] = (rx_buf[1]<<8)>>8;
```

- décalage vers la gauche → amène le bit de signe au 32ème bit
- décalage vers la droite → restore la valeur

Après avoir fait ça, on peut appliquer la division sur le signal et vérifier que tout fonctionne bien.



Le signal violet est bien deux fois plus petit que le signal jaune.

Augmentation de la fréquence du signal d'entrée

Lorsqu'on augmente la fréquence du signal d'entrée, on observe que **le signal de sortie diminue en amplitude**.

Ceci est dû à l'échantillonnage du signal d'entrée. Lorsque la fréquence du signal se rapproche de la fréquence d'échantillonnage de l'ADC (48 kHz), on perd beaucoup d'information. L'amplitude du signal de sortie s'en retrouve réduite.

4. Mise en place de buffers tampons

Pour mettre en place un filtre, on a besoin de plusieurs valeurs à traiter. Avec la méthode que nous avons vu dans la partie précédente, on n'a qu'une seule valeur à disposition. Il va donc falloir changer de méthode.

L'objectif va être de traiter le signal en "morceaux".

- entrée → récupère les échantillons envoyés par l'ADC et les stocke dans un buffer
- traitement → applique un filtre sur le buffer contenant un ensemble d'échantillons
- sortie → restitue les valeurs traitées à la même fréquence que l'ADC

On aura donc 3 buffers qui joueront simultanément ces 3 rôles.

	...	n	n+1	n+2	n+3	...
Buffer 1	...	Entrée	Traitement	Sortie	Entrée	...
Buffer 2	...	Sortie	Entrée	Traitement	Sortie	...
Buffer 3	...	Traitement	Sortie	Entrée	Traitement	...

On crée donc les différents buffers :

```
float* buffers_ptrs[3];
float buffers[3][MAX_BUFFER_SIZE];
```

Remarque : Par souci d'optimisation, on ne va pas bouger tous les tableaux lorsque les buffers changent de rôles. A la place, nous avons la liste "buffers_ptrs" qui stocke les pointeurs de tous les buffers. La position du pointeur dans la liste nous donne le rôle qu'il a actuellement.

Lorsque l'on veut passer à l'étape suivante, on fait appel à la fonction "rotate_buffers" :

```
void rotate_buffers(void){
    float* temp = buffers_ptrs[OUTPUT_BUFFER_INDEX];
    buffers_ptrs[OUTPUT_BUFFER_INDEX] = buffers_ptrs[PROCESSING_BUFFER_INDEX];
    buffers_ptrs[PROCESSING_BUFFER_INDEX] = buffers_ptrs[INPUT_BUFFER_INDEX];
    buffers_ptrs[INPUT_BUFFER_INDEX] = temp;
}
```

Rotation des buffers

La rotation des buffers s'effectue à chaque interruption causée par le signal externe. On reprend donc la fonction IRQP1_isr :

```
void IRQP1_isr ( int sig_num )
{
    static int IT_register;
    IT_register = *pDAI_IRPTL_H;

    //tester si c'est bien DAI_P1 qui est à l'origine de l'interruption DAI
    if ((IT_register & SRU_EXTMISC1_INT) != 0)
    {
        rotate_buffers();
        final_input_values_count = input_values_count;
        input_values_count = 0;
        start_processing = 1;
    }

    return;
}
```

- la variable "input_value_count", count le nombre d'échantillons reçus depuis le dernier switch de buffers
- "start_processing" est un flag qui indique au programme principal de commencer le traitement du buffer de traitement.

Entrée et Sortie

Dans le code d'interruption de l'ADC :

- récupération de rx
- passage sur 32 bits
- ajout de la valeur dans le buffer d'entrée
- envoi sur tx des valeurs dans le buffer de sortie

```
void ReceptionISR(int sig_int)
{
    buffers_ptrs[INPUT_BUFFER_INDEX][input_values_count] = (float)((rx_buf[0]<<8)>>8);
    tx_buf[0]= (int)buffers_ptrs[OUTPUT_BUFFER_INDEX][input_values_count];
    input_values_count++;
}
```

En fait, on envoie une valeur en sortie dès qu'on reçoit une nouvelle valeur.

Traitement

Le traitement du buffer de traitement est assuré par la boucle principale.

```
while(1){
    if(start_processing == 1){
        // computations on the processing buffer
    }
}
```

Le signal "start_processing" est mis à 1 lors du switch de buffer.

Pour l'instant, il n'y a pas de processing.

A cette étape, l'application talk-through fonctionne toujours, mais les buffers tampons sont en place.

5. Traitement d'un signal carré

Dans cette partie, nous allons appliquer un filtre lisseur sur un signal carré.

Nous commençons par récupérer la fonction "Lisseur1D" que nous avons écrit dans le cadre des tutoriels :

$$y_n = (1 - \gamma) \cdot x_n + \gamma \cdot y_{n-1}$$

```
void Lisseur1D(float* tab, float gamma, int N)
{
    int i;

    float sn_1 = tab[0];
    for (i=1; i<N; i++){
        tab[i] = sn_1 = (1-gamma)*tab[i] + gamma*sn_1;
    }

    sn_1 = tab[N-1];
    for (i=1; i<N; i++) {
        tab[N-i-1] = sn_1 = (1-gamma)*tab[N-i-1] + gamma*sn_1;
    }
}
```


Remarque : Ce filtre est appelé "Filtre de Deriche". Dans le code, nous avons combiné une version causale et une version non-causale, pour obtenir une sortie symétrique pour le signal carré (cf tutoriels).

On appelle donc cette fonction dans notre boucle principale :

```
while(1){
    if(start_processing == 1){
        // computations on the processing buffer
        Lisseur1D(buffers_ptrs[PROCESSING_BUFFER_INDEX], gamma, final_input_values_count);
        start_processing = 0;
    }
}
```

- gamma est fixé à 0.5

On compile le code, et on l'envoie sur la carte.

Grâce au GBF, on envoie un signal créneau sur l'entrée.



On observe bien en sortie un lissage du signal créneau d'entrée. Le lissage est bien symétrique.

Remarque : Dans la première version du filtre que nous avons testé, nous avions une discontinuité entre les morceaux de signaux traités séparément. Cette discontinuité était due à l'initialisation de y_{n-1} à 0. Pour remédier à ce problème, nous avons sauté le calcul du filtrage pour la première valeur de chaque morceau.

Lorsqu'on augmente la fréquence du signal d'entrée, on diminue le nombre de points par morceaux (la fréquence de l'ADC reste fixe, et le temps entre chaque switch de buffer diminue).

La qualité du filtrage est alors affectée. Un trop petit nombre d'échantillon ne permet pas de lisser le signal.

6. Interruption Bouton

Dans cette partie, on s'intéresse à l'utilisation des interruptions bouton. Tout le principe est expliqué dans le TP4.

Le bouton que nous voulons utiliser est relié à un pin nommé "FLAG1":

Push Button Reference Designator	DSP Pin
SW1	FLAG1
SW2	FLAG2
SW3	DAI19
SW4	DAI20

Remarque : Nous aurions aussi pu utiliser les boutons connectés aux ports DAI, mais ça aurait été plus compliqué.

On commence par activer l'interruption IRQ1 (venant du SW1)

```
*pSYSCCTL |= IRQ1EN;
```

Puis, on définit que l'interruption sera active sur front

```
asm("bit set mode2 IRQ1E;");
```

Enfin, on définit la fonction de callback

```
interrupt(SIG_IRQ1, Button_Interrupt_Handler);
```

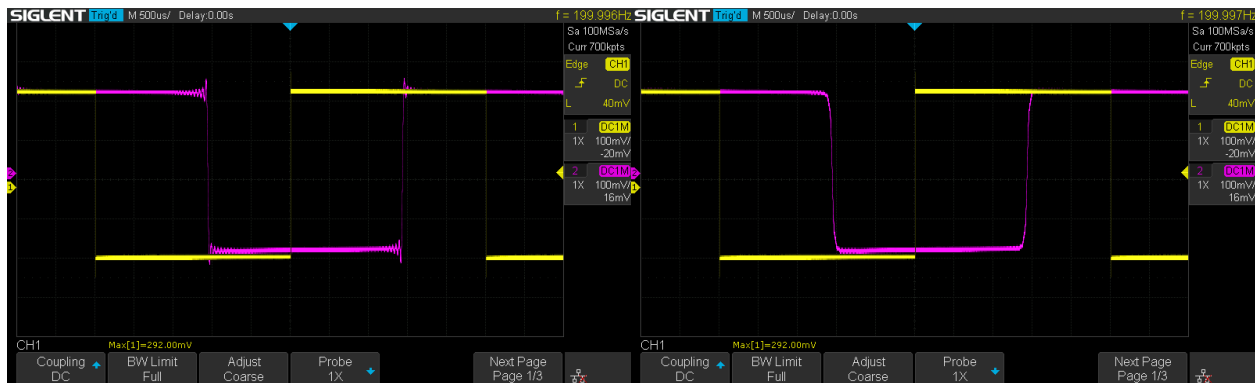
Remarques:

- le registre MODE 2 n'est pas accessible via pointeurs, on est obligés d'utiliser de l'assembleur
- Pour éviter d'avoir à changer les bits des registres à la main, on utilise les macros fournies par les librairies.

La fonction de callback du bouton est la suivante :

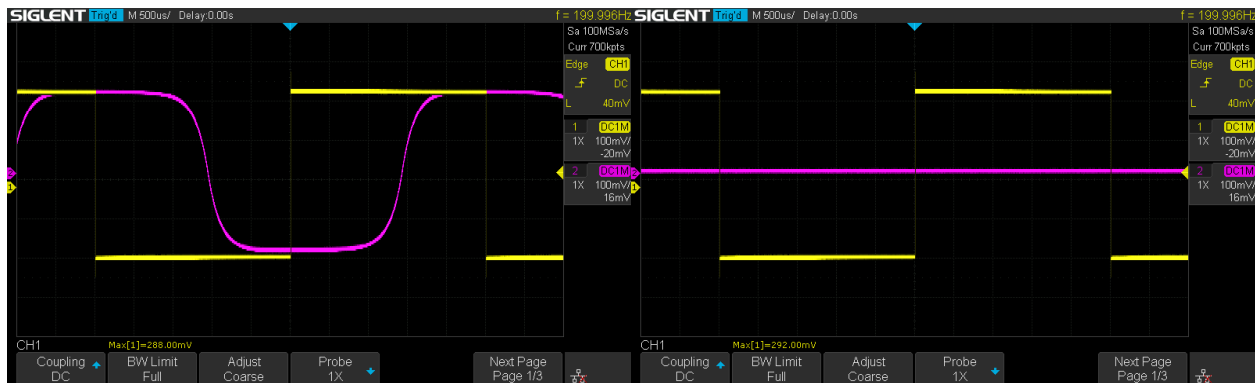
```
void Button_Interrupt_Handler(int sig_num){
    gamma = gamma + 0.125;
    if (gamma > 1){
        gamma = 0;
    }
    return;
}
```

On peut appuyer plusieurs fois sur le bouton pour faire varier gamma et regarder sur l'oscilloscope l'effet du changement:



$\gamma = 0$

$\gamma = 0.250$



$\gamma = 0.875$

$\gamma = 1$

- Augmenter γ accentue la courbure (le lissage) du signal de sortie.

Remarque: Pour $\gamma = 1$, la formule du filtre devient $y_n = y_{n-1}$. Or $y_0 = 0$. C'est pourquoi on obtient un signal nul en sortie.

7. Version assembleur du filtre

La version assembleur du filtre de Deriche a été écrite en TD. Nous allons plutôt utiliser le compilateur de VisualDSP pour générer le code assembleur de notre fonction "Lisseur1D".

On se rend donc dans les fichiers d'installation de VisualDSP, dans lesquels se trouve un fichier exécutable nommé "cc21k.exe". Dans le CMD, on lance cet exécutable en spécifiant le fichier à compiler (dans notre cas, c'est un fichier C contenant seulement la fonction "Lisseur1D").

On obtient en sortie un fichier assembleur que l'on peut examiner:

- Au début du fichier on retrouve le label de la fonction "Lisseur1D"

```

.file "liss1d.c";

// Compilation time: Wed Jan 07 16:51:11 2026

// Compiler options: -c -S -O

// Architecture: ADSP-21060

// Silicon revision: 3.1

.section/pm/DOUBLE32 seg_pmco;

.epctext:
2 references
_Lisseur1D:
.LN_Lisseur1D:
//-----

```

- On peut voir la boucle principale de calcul de la sortie

```

    lcntr=r0, do(pc,.P1L12-1)until lce;

.P1L11:

    F9=F8*F2, r4=dm(i0,m6);
    F12=F1*F4, F2=F9+F12, dm(i4,m6)=r2;
    //end loop .P1L11;
//-----

```

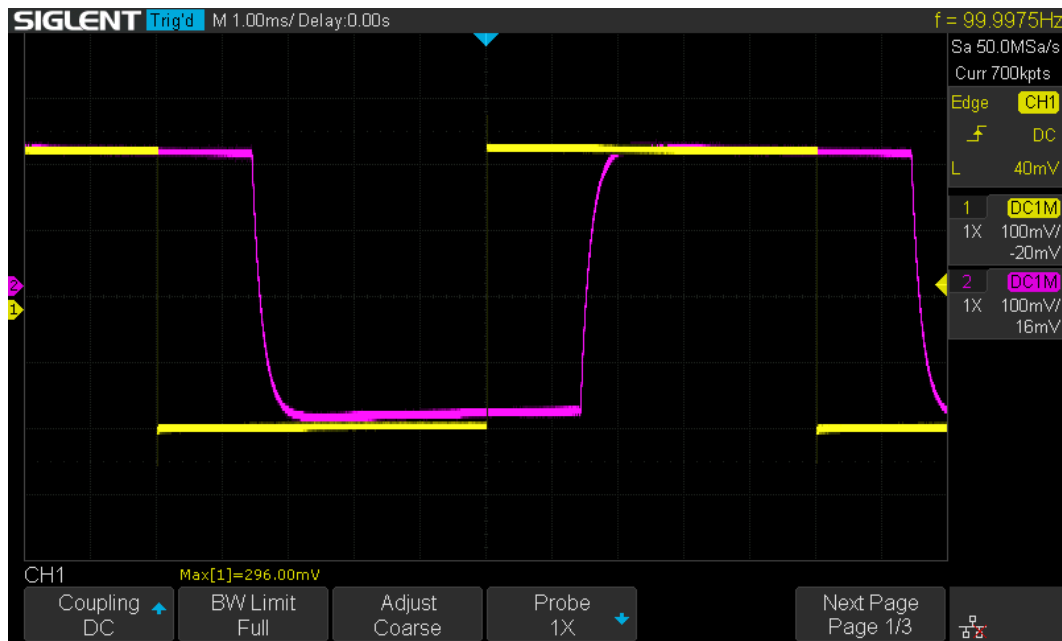
Remarque : le compilateur a été lancé en mode "optimisation". La boucle principale est donc réduite à 2 cycles. Le processeur tire avantage des opérations qu'il peut faire en parallèle.

Tout le reste du fichier correspond à de la mise en place pour les arguments, pour les sorties, etc...

On peut alors retourner dans le projet VisualDSP et ajouter le fichier Lisseur1D.asm

On retire momentanément la fonction Lisseur1D déjà présente en C (sous peine de conflit). On ajoute toutefois un prototype à la fonction. De cette manière, le code C ira directement chercher la fonction Lisseur1D dans le fichier ASM, mais l'appel restera le même.

On peut donc compiler et envoyer le code.



- Pour faciliter la compréhension du code assembleur, nous avons retiré la partie anti-causale du filtre. On a donc une sortie non symétrique
- On retrouve quand même un lissage du signal créneau

8. Mesure de la fréquence d'entrée

Pour mesurer la fréquence du signal d'entrée, nous allons utiliser la fréquence d'échantillonnage. Nous savons que celle-ci est fixe (48 kHz). De plus, nous savons que l'interruption de switch de buffer arrive exactement à la fréquence du signal.

Le nombre d'échantillons dans un morceau est donc $N_{ech} = \frac{f_{sampling}}{f_{switch}} \Leftrightarrow f_{switch} = \frac{f_{sampling}}{N_{ech}}$

On peut donc écrire le code suivant :

```
// frequency computation
chunk_time = ((float)final_input_values_count)/48000;
frequency = (int) (1/chunk_time);
AfficheLEDs(frequency);
```

La fréquence du signal d'entrée est donc affichée en binaire sur les leds de la carte.

Conclusion

Pendant ce TP, nous avons :

- Mis en place des interruptions sur la carte (signal externe, bouton)
- Réalisé un pipe-line complet de traitement de signal
- Analysé le code assembleur du filtre de Deriche